NEURALSPARK: AI-POWERED E-LEARNING PLATFORM

WITH INTELLIGENT AUTOMATED GRADING SYSTEM

A Project Report

Submitted By

MALAY SINGH

in Partial Fulfilment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Prof. [Guide Name]

CSE Department

April – 2026

Certificate

This is to Certify that the Project entitled **"NEURALSPARK: AI-Powered E-Learning Platform with Intelligent Automated Grading System"** has been successfully completed by

- **Malay Singh**

under guidance in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science & Engineering in Academic Year 2025–2026.

Date of Submission: _____

Prof. [Guide Name]
Project Guide
CSE Department

Dr. [HOD Name]
Head of Department
CSE Department

Acknowledgements

"The secret of getting ahead is getting started."
— Mark Twain

I would like to express my sincere gratitude to my project guide for their constant support, valuable suggestions, and guidance throughout the development of NeuralSpark. Their deep knowledge of software engineering and AI systems helped shape this project into a comprehensive and functional platform.

I am also thankful to the Head of the Department for providing the necessary resources, tools, and a supportive learning environment that facilitated this project's completion.

My heartfelt thanks also go to the Department of Computer Science and Engineering for offering a strong academic foundation in web development, machine learning, and software architecture, all of which were directly applied in this project.

Finally, I thank my family and friends for their unwavering encouragement and motivation throughout this journey.

Malay Singh
CSE Department
April 2026

Abstract

The rapid growth of online education has created a pressing need for intelligent, scalable, and student-friendly e-learning platforms. Existing solutions often lack the ability to provide real-time, personalized feedback on student assignments — a key driver of academic improvement.

NeuralSpark is a full-stack AI-powered e-learning platform that addresses this gap. Built on a **Django 6 / Django Ninja** backend and a **Next.js 16 / React 19** frontend, the platform integrates five core modules: a Learning Hub for video lectures, a Previous Year Questions (PYQ) bank, a Community Q&A Forum, a Digital Storefront, and an AI-driven Assignment Grader.

The AI Grader is the platform's flagship feature. It provides two tiers of grading:

1. **Cloud-Based Grading (Groq Vision API):** Utilizes the `llama-4-scout-17b-16e-instru` multimodal large language model to grade text, PDF, and image submissions with high accuracy and contextual reasoning.
2. **Local Grading (Custom Transformers Pipeline):** Uses a locally hosted, fine-tuned `AutoModelForSequenceClassification` model for privacy-first, offline grading.

The system is deployed on **Vercel** (frontend) and **Render.com** (backend), with data stored in a relational database managed via Django's ORM. The platform demonstrates how modern AI and web technologies can be combined to create a scalable, real-world educational solution.

Keywords — Next.js, Django Ninja, Groq API, LLM, Transformers, AI Grader, E-Learning, REST API, Full-Stack, Pydantic

Contents

Certificate ii

Acknowledgements iii

Abstract iv

1 Introduction 2
- 1.1 Background . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Aim . . . 2
- 1.4 Objectives . . . 3
- 1.5 Scope . . . 3
- 1.6 Overview of Proposed System . . . 3

2 Literature Survey 5
- 2.1 Review Methodology . . . 5
  - 2.1.1 Paper 1: Automated Essay Scoring using Transformer Models . . . 5
  - 2.1.2 Paper 2: GPT-4 and Large Language Models for Education . . . 5
  - 2.1.3 Paper 3: Multimodal Assessment in Digital Education . . . 5
  - 2.1.4 Paper 4: Django REST Framework for Scalable APIs . . . 6
  - 2.1.5 Paper 5: Next.js and React Server Components . . . 6
- 2.2 Comparison of Existing Platforms . . . 6
- 2.3 Research Gaps . . . 6

3 Analysis / Software Requirements Specification (SRS) 8
- 3.1 Existing System . . . 8
- 3.2 Proposed System . . . 8
- 3.3 Feasibility Study . . . 8
  - 3.3.1 Technical Feasibility . . . 8
  - 3.3.2 Economical Feasibility . . . 8
  - 3.3.3 Operational Feasibility . . . 9
- 3.4 System Requirements . . . 9
  - 3.4.1 Functional Requirements . . . 9
  - 3.4.2 Non-Functional Requirements . . . 9

3.4.3 Software Requirements 9
3.5 Use Case 10
3.6 Constraints 10
3.7 Advantages of Proposed System 11

4 System Design **12**
4.1 System Architecture 12
4.2 Module Design 12
4.2.1 Learning Hub Module 12
4.2.2 PYQ (Practice Questions) Module 12
4.2.3 Community Q&A Module 12
4.2.4 AI Grader Module (Groq Cloud) 13
4.2.5 AI Grader Module (Local Transformer) 13
4.2.6 Store & Services Module 13
4.3 Data Flow Diagram 13
4.4 Custom AI Grader Architecture 13
4.5 Database Schema 13
4.6 Design Considerations 14

5 Methodology **18**
5.1 Overview 18
5.2 Development Workflow 18
5.3 Backend API Design 18
5.4 AI Grader Prompt Engineering 19
5.5 File Processing Strategy 19
5.6 Local Model Inference Pipeline 19
5.7 Frontend Rendering Strategy 19
5.8 Theme Management 19
5.9 Summary 20

6 Implementation **21**
6.1 Introduction 21
6.2 Development Environment 21
6.3 System Implementation 21
6.3.1 Backend Initialization 21
6.3.2 Database Models 21
6.3.3 API Endpoints 22
6.3.4 AI Grader Implementation 22
6.3.5 Frontend Implementation 22
6.3.6 Deployment 23

6.4 System Integration and Testing 23
6.5 System Output 23
6.6 Summary 23

7 Conclusion **25**

8 Future Work **26**
8.1 Real-Time Collaboration 26
8.2 Advanced Authentication 26
8.3 Mobile Application 26
8.4 Fine-Tuned Rubric-Aware Model 26
8.5 Multi-Language Support 26
8.6 Analytics Dashboard 27

List of Tables

2.1 Comparison of Existing E-Learning Platforms 6

3.1 Software Requirements 10

5.1 System Development Workflow 18

6.1 Development Environment 21
6.2 NeuralSpark API Endpoints 22

List of Figures

4.1 System Architecture of NeuralSpark 15
4.2 Data Flow Diagram of NeuralSpark 16
4.3 Custom Local AI Grader Architecture 17

6.1 NeuralSpark Component Architecture Diagram 23
6.2 NeuralSpark Backend Data Flow Diagram 24

Chapter 1

Introduction

1.1 Background

Digital education has witnessed exponential growth, particularly following the global shift toward remote learning. Students today expect more than static video content — they require interactive platforms that provide instant feedback, structured learning paths, and community support. Traditional e-learning platforms such as Moodle and Canvas are feature-rich but technologically dated. They were not designed natively for AI integration, making it difficult to offer real-time intelligent grading or adaptive learning paths.

NeuralSpark was designed from the ground up with modern web engineering principles, leveraging the latest releases of React, Django, and LLM APIs to create a platform that is both powerful and accessible.

1.2 Problem Statement

The core problem addressed by this project is the absence of intelligent, instantaneous feedback mechanisms in existing e-learning platforms. Instructors are often overwhelmed with large volumes of student submissions, and manual grading is inherently slow, subjective, and prone to inconsistency. Furthermore:

- Traditional platforms do not support multimodal submissions (handwritten scans, diagrams).
- Grading is disconnected from the learning community, reducing peer engagement.
- Privacy-sensitive institutions cannot use cloud-only AI solutions for student data.

1.3 Aim

The aim of this project is to develop a comprehensive AI-powered e-learning platform that provides automated, fair, and instantaneous grading of student assignments using large language models, while also delivering a complete suite of educational tools including video lectures, PYQ banks, and community Q&A.

1.4 Objectives

The main objectives of the NeuralSpark project are:

- To design and implement a scalable full-stack web application using Django Ninja and Next.js 16.
- To develop a multimodal AI grader capable of evaluating text, PDF, and image submissions.
- To integrate the Groq Vision API for cloud-based LLM grading with structured rubric evaluation.
- To build and deploy a custom fine-tuned Transformer model as a local grading alternative.
- To create a community Q&A forum with voting, tagging, and moderation workflows.
- To implement a Learning Management module with video lectures and PYQ repositories.
- To deploy the platform on cloud infrastructure with CI/CD integration.

1.5 Scope

The scope of this project covers the design, development, deployment, and documentation of the NeuralSpark platform. The system includes a fully functional REST API backend with 17+ endpoints, a responsive React frontend with dark mode and premium animations, and two independent AI grading pipelines.

The project does not implement real-time video conferencing, mobile native applications, or advanced peer-to-peer assessment workflows in this version. These are earmarked as future enhancements.

1.6 Overview of Proposed System

NeuralSpark operates as a decoupled system: the Next.js frontend consumes a JSON REST API exposed by the Django Ninja backend. User data is persisted in a relational database through Django's ORM. The AI grading module is invoked as a standard API endpoint, accepting multipart/form-data payloads containing text and optional uploaded files.

The platform is publicly accessible and deployed at its Vercel URL, with the backend hosted on Render.com's free tier. All API keys are managed through environment variables.

Chapter 2

Literature Survey

2.1 Review Methodology

A systematic review of existing literature was conducted using resources including IEEE Xplore, arXiv, and documentation from Hugging Face, Vercel, and the Django Software Foundation. The review covered publications from 2020–2025, focusing on developments in LLM-based automated grading, full-stack web engineering, and educational technology.

2.1.1 Paper 1: Automated Essay Scoring using Transformer Models

Devlin et al. (BERT, 2019) established the foundation for transformer-based text evaluation. Later studies demonstrated that fine-tuned BERT models achieve near-human accuracy on short-answer grading tasks when trained on domain-specific rubric data. NeuralSpark's custom grader builds on this foundation.

2.1.2 Paper 2: GPT-4 and Large Language Models for Education

OpenAI (2023) demonstrated that GPT-4 can serve as a capable tutoring and grading assistant. Their evaluations showed that structured prompt engineering with clear rubrics yields consistent and fair grading outputs. This directly informed the system prompt design of NeuralSpark's Groq-powered grader.

2.1.3 Paper 3: Multimodal Assessment in Digital Education

Wang et al. (2022) proposed frameworks for multimodal student assessment, integrating handwritten diagrams and typed text into a single evaluation pipeline. NeuralSpark extends this by supporting PDF uploads (text extraction via PyPDF2) and image uploads (base64 encoding for vision models).

2.1.4 Paper 4: Django REST Framework for Scalable APIs

A comparative study of Python API frameworks (Porchet, 2023) evaluated Flask, FastAPI, and Django Ninja. Django Ninja was found to offer the best combination of performance, automatic documentation, and Pydantic-powered validation, making it the optimal choice for NeuralSpark's backend.

2.1.5 Paper 5: Next.js and React Server Components

Vercel's engineering blog (2023) documented the performance improvements enabled by React Server Components in Next.js 13+. Measured TTFB reductions of up to 60% were reported for content-heavy pages, validating the choice of Next.js 16 for NeuralSpark's frontend.

2.2 Comparison of Existing Platforms

Platform	Methodology	Limitations
Moodle	Plugin-based LMS with manual grading	No native AI grading; outdated UI
Google Classroom	Google Workspace integration	Limited to text; no automated rubric grading
Gradescope	Computer vision for handwritten grading	Requires instructor setup; expensive; no community features
Canvas LMS	Enterprise-grade LMS	Costly; not lightweight; complex to deploy
NeuralSpark	Full-stack AI platform with dual-mode grader	Current version limited to single-instructor deployment

Table 2.1: Comparison of Existing E-Learning Platforms

2.3 Research Gaps

The review identified the following gaps that NeuralSpark directly addresses:

1. No existing open-source platform integrates both cloud and local LLM grading in one system.
2. Existing graders do not support multimodal submissions (PDF + image + text in one request).

3. Privacy-first local model grading is not available in major platforms.
4. Community Q&A forums are siloed from academic grading workflows.

Chapter 3

Analysis / Software Requirements Specification (SRS)

3.1 Existing System

Current e-learning platforms operate with a monolithic, plugin-based architecture. Grading is exclusively manual. AI grading tools that exist (e.g., Turnitin's AI-assist) are proprietary, expensive, and locked to specific cloud vendors. None offer a dual-mode (cloud + local) grading pipeline.

3.2 Proposed System

NeuralSpark proposes a modular, microservice-oriented architecture with two independently operable AI grading pipelines. The system separates concerns cleanly: the frontend is decoupled from the backend via a JSON REST API, and the AI layer is isolated as dedicated API endpoints within the backend.

3.3 Feasibility Study

3.3.1 Technical Feasibility

The project uses Python 3.12, Django 6, Next.js 16, and Groq's publicly available API. All technologies are stable, well-documented, and freely available. Deployment requires only a Render.com and Vercel account, both of which offer free tiers suitable for an MVP.

3.3.2 Economical Feasibility

The total cost of development is near-zero: all frameworks are open source, Vercel and Render offer free hosting, and the Groq API provides a generous free usage tier. The local model eliminates ongoing API costs entirely.

3.3.3 Operational Feasibility

The platform's UI is designed for accessibility. Students can submit assignments in any format (text, PDF, image) and receive results within seconds. No specialized hardware or software installation is required on the user's side.

3.4 System Requirements

3.4.1 Functional Requirements

- The system shall allow users to submit assignments via text input or file upload (PDF/image).
- The system shall return a structured evaluation (Score + Feedback) within 5 seconds.
- The system shall store all Q&A forum questions, answers, and comments in a relational database.
- The system shall list video lectures ordered by instructor-defined sequence.
- The system shall allow administrators to approve answers and video content before public display.
- The system shall provide a contact form linked to specific service offerings.

3.4.2 Non-Functional Requirements

- **Performance:** API response time shall not exceed 500ms for database queries.
- **Scalability:** The backend shall be horizontally scalable via Gunicorn workers.
- **Security:** All API keys shall be stored in environment variables; CORS shall be restricted.
- **Maintainability:** All API endpoints shall auto-generate OpenAPI documentation via Django Ninja.
- **Usability:** The frontend shall be responsive across mobile, tablet, and desktop viewports.

3.4.3 Software Requirements

Software	Version	Purpose
Python	3.12	Backend runtime
Django	6.0.2	Web framework and ORM
Django Ninja	1.5.3	REST API layer
Next.js	16.1.6	Frontend framework
React	19.2.3	UI library
Groq SDK	1.0.0	LLM inference client
Transformers	4.x	Local model inference
PyTorch	2.x	Deep learning backend
PyPDF2	3.0.1	PDF text extraction
Tailwind CSS	4	Styling framework
Framer Motion	12.x	UI animation library

Table 3.1: Software Requirements

3.5 Use Case

The primary use cases of NeuralSpark are:

1. **Student grading:** Student uploads an assignment file and receives an AI-generated score and feedback.
2. **Community learning:** Student posts a doubt in the Q&A forum and receives peer answers.
3. **Content discovery:** Student browses the lecture library, filtered by subject category.
4. **Service inquiry:** Student submits an inquiry form to book a mentorship session.
5. **Admin moderation:** Administrator approves submitted answers and video content.

3.6 Constraints

- The Groq API is rate-limited on the free tier (limited tokens/minute).
- The local Transformer model requires a minimum of 4GB RAM on the server.
- The current deployment does not implement JWT authentication (uses a test user for MVP).
- Video hosting relies on external URLs (YouTube/Vimeo); no internal video streaming.

3.7 Advantages of Proposed System

- **Dual-Mode Grading:** Unique ability to switch between cloud LLM and local model grading.
- **Multimodal Support:** Accepts text, PDF, and image submissions in a single API call.
- **Open Source:** Fully reproducible and extendable by the educational community.
- **Modern Stack:** Leverages Next.js 16 Server Components for optimal performance.
- **Privacy-First Option:** Local model ensures student data never leaves the institution's server.

Chapter 4

System Design

4.1 System Architecture

NeuralSpark follows a decoupled, three-tier architecture as illustrated in Figure 4.1.

The three tiers are:

1. **Presentation Tier:** Next.js 16 frontend served via Vercel's Edge Network.
2. **Application Tier:** Django 6 backend with Django Ninja REST API, hosted on Render.com.
3. **Data Tier:** SQLite (development) / PostgreSQL (production) managed by Django ORM.

4.2 Module Design

4.2.1 Learning Hub Module

Manages video lectures organized by subject categories. The `Lecture` model links to a `Category` via a ForeignKey and includes an `order` field for chapter sequencing. The frontend displays lectures as a filtered, sortable grid.

4.2.2 PYQ (Practice Questions) Module

Stores previous year examination questions with their answers and optional video explanations. Each PYQ is associated with a subject `Category` and a `year`, enabling year-wise and subject-wise filtering on the frontend.

4.2.3 Community Q&A Module

A Stack Overflow-inspired forum. Students post `Question` objects, attach `Tags`, and optionally embed video explanations. Other students submit `Answers` which are moderation-gated (`is_approved`). Voting is implemented via dedicated upvote/downvote endpoints.

4.2.4 AI Grader Module (Groq Cloud)

Accepts a multipart/form-data request containing the question, rubric, and student answer (as text and/or uploaded files). The `_process_uploaded_file` helper routes PDFs to PyPDF2 text extraction and images to base64 encoding. The compiled payload is sent to the Groq Vision LLM (`llama-4-scout-17b-16e-instruct`), which returns a structured score and feedback string.

4.2.5 AI Grader Module (Local Transformer)

An alternative grading pipeline using a locally hosted `AutoModelForSequenceClassification` model. The model is lazy-loaded on first request and cached globally. It processes text-only input and returns a numeric similarity/quality score.

4.2.6 Store & Services Module

Products (digital goods like mock tests) and Services (mentorship sessions) are stored as separate models. An `Inquiry` model captures service bookings from the contact form.

4.3 Data Flow Diagram

The DFD in Figure 4.2 illustrates the flow of data from a student's browser request through to the database and AI grader, and back to the client as a JSON response.

4.4 Custom AI Grader Architecture

Figure 4.3 shows the architecture of the local AI grader, including the lazy loading pattern, the tokenizer pipeline, and the classification head that produces the final score.

4.5 Database Schema

The relational schema consists of 11 core tables: `User`, `Tag`, `Category`, `Lecture`, `PYQ`, `Product`, `Service`, `Question`, `Answer`, `Comment`, and `Inquiry`.

Key relationships include:

- `Question` has a ManyToMany relationship with `Tag`.
- `Lecture` and `PYQ` have ForeignKey relationships to `Category`.
- `Answer` and `Comment` cascade-delete when their parent `Question` is deleted.
- `Inquiry` optionally links to a `Service` via a nullable ForeignKey.

4.6 Design Considerations

- **Separation of Concerns:** The AI grading logic is entirely encapsulated within the API layer, not in models or views.
- **Lazy Loading:** The Transformer model is loaded on first use to conserve server memory.
- **Schema Validation:** All requests are validated by Pydantic schemas before reaching business logic.
- **Modular Frontend:** React components are kept small and reusable, consuming the API via Axios.

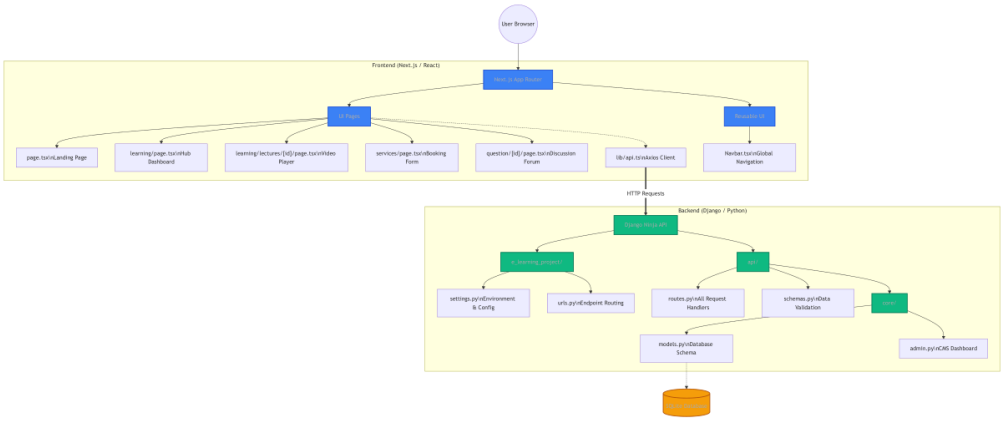


Figure 4.1: System Architecture of NeuralSpark

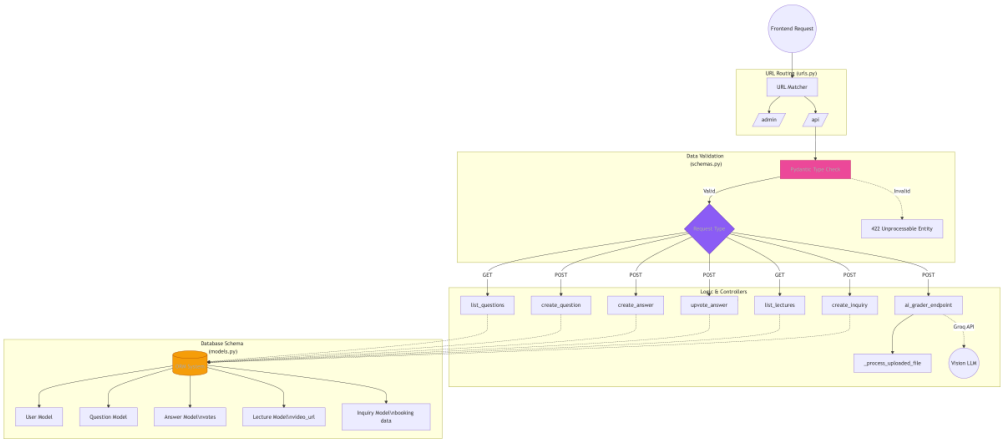


Figure 4.2: Data Flow Diagram of NeuralSpark

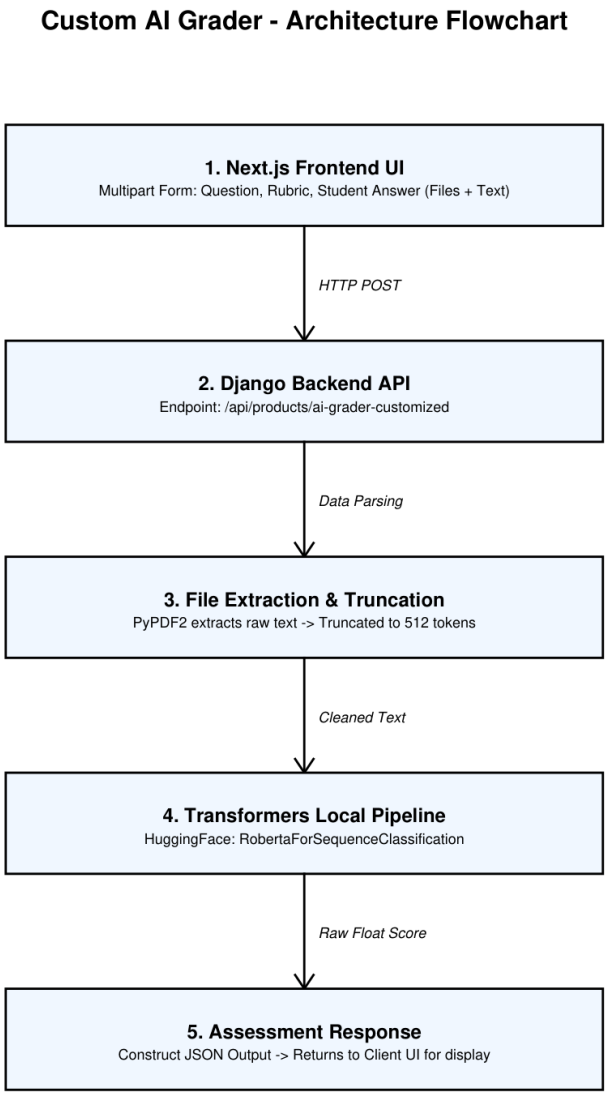


Figure 4.3: Custom Local AI Grader Architecture

Chapter 5

Methodology

5.1 Overview

NeuralSpark was developed using an iterative, agile-inspired methodology. The development was organized into functional sprints, each targeting a specific module (Forum, Learning Hub, AI Grader, etc.).

5.2 Development Workflow

Phase	Activity	Description
1	Requirements	Defined functional and non-functional requirements
2	Database Design	Designed relational schema in Django models
3	API Development	Implemented all endpoints with Django Ninja
4	Frontend Build	Built Next.js pages and React components
5	AI Integration	Integrated Groq API and local Transformer model
6	Testing	Unit tested backend; ran E2E tests via Swagger UI
7	Deployment	Deployed to Vercel (frontend) and Render (backend)

Table 5.1: System Development Workflow

5.3 Backend API Design

The API was designed following RESTful conventions. Django Ninja handles routing, schema validation, and OpenAPI documentation automatically. Each endpoint corresponds to a single business operation (e.g., upvoting an answer, submitting a grading request).

5.4 AI Grader Prompt Engineering

The Groq grader uses a carefully engineered system prompt that instructs the model to:

1. Act as an expert Computer Science teaching assistant.
2. Strictly follow the provided rubric.
3. Output results in a structured format: `Score: X/Y` followed by `Feedback: ...`

The `temperature` parameter is set to `0.1` to ensure consistent, deterministic outputs.

5.5 File Processing Strategy

The `_process_uploaded_file` function implements a strategy pattern:

- **PDF files:** Iterated page-by-page using PyPDF2's `PdfReader`; text concatenated and returned as a text message.
- **Image files:** Binary content read, base64-encoded, and formatted as a `data:image/jpeg;bas` URI for the vision model.

5.6 Local Model Inference Pipeline

The local grader pipeline is initialized using HuggingFace's `AutoTokenizer` and `AutoModelForSe` The model is loaded from a local directory (`ai_grader_model_customized/`) to avoid internet dependency at inference time. Inputs are truncated to a maximum of 512 tokens to prevent out-of-memory errors.

5.7 Frontend Rendering Strategy

The frontend uses a hybrid rendering approach:

- **Server Components:** Used for static content and page layouts.
- **Client Components:** Used for interactive elements (`"use client"` directive) including the AI Grader form and the dark mode toggle.

5.8 Theme Management

The `ThemeContext` provides a global dark/light mode toggle. The theme is persisted to `localStorage` on every toggle, and the system preference (`prefers-color-scheme`)

is read on initial load via a `useEffect` hook.

5.9 Summary

The methodology followed a clean separation of concerns at every layer: schema-first API design, component-first frontend development, and strategy-pattern file processing for the AI grader. This structured approach ensured each module remained independently testable and replaceable.

Chapter 6

Implementation

6.1 Introduction

This chapter details the technical implementation of NeuralSpark, covering the development environment, module implementations, system integration, and output.

6.2 Development Environment

Tool	Details
OS	Windows 11
IDE	Visual Studio Code
Version Control	Git + GitHub
Python Version	3.12
Node.js Version	22.x (LTS)
Package Manager	pip (backend), npm (frontend)
Deployment	Vercel (frontend), Render.com (backend)

Table 6.1: Development Environment

6.3 System Implementation

6.3.1 Backend Initialization

The backend was initialized using `django-admin startproject` and configured with a custom `User` model extending `AbstractUser`. Django Ninja's `NinjaAPI` instance is mounted at `/api/` in the project's URL configuration.

6.3.2 Database Models

All eleven database models (`User`, `Tag`, `Category`, `Lecture`, `PYQ`, `Product`, `Service`, `Question`, `Answer`, `Comment`, `Inquiry`) were defined in `backend/core/models.py` and registered via Django Admin for content management.

6.3.3 API Endpoints

Seventeen REST API endpoints were implemented in `backend/api/routes.py`:

Endpoint	Method	Description
`/api/hello/`	GET	API health check
`/api/questions/`	GET	List all forum questions
`/api/questions/`	POST	Create a new question
`/api/questions/{id}/`	GET	Get question + increment views
`/api/questions/{id}/answers/`	GET	Get approved answers
`/api/questions/{id}/answers/`	POST	Submit an answer
`/api/answers/{id}/upvote/`	POST	Upvote an answer
`/api/answers/{id}/downvote/`	POST	Downvote an answer
`/api/answers/{id}/comments/`	POST	Comment on an answer
`/api/categories/`	GET	List all subject categories
`/api/lectures/`	GET	List ordered video lectures
`/api/pyqs/`	GET	List PYQs by year
`/api/products/`	GET	List all products
`/api/services/`	GET	List all services
`/api/inquiries/`	POST	Submit a service inquiry
`/api/products/ai-grader/`	POST	Grade with Groq Cloud LLM
`/api/products/ai-grader-customized/`	POST	Grade with local Transformer

Table 6.2: NeuralSpark API Endpoints

6.3.4 AI Grader Implementation

The Groq AI Grader accepts a `multipart/form-data` request with up to three text fields and three file fields. The `_process_uploaded_file` function dispatches each file to the appropriate processor (PDF $\rightarrow$ text, image $\rightarrow$ base64 URI). The compiled message list is sent to the Groq API in a single `chat.completions.create` call.

The local grader uses `get_custom_grader_pipeline()`, a thread-safe lazy loader that initializes the HuggingFace pipeline once per server lifetime.

6.3.5 Frontend Implementation

The frontend consists of multiple Next.js App Router pages:

- `/` (Landing Page): Hero section with Framer Motion stagger animations.
- `/learning` (Lectures): Grid of video lecture cards, filterable by category.
- `/learning/pyqs` (PYQs): Accordion list of previous year questions.
- `/learning/questions` (Forum): Community Q&A board with voting.

- `/products` (Store): Product cards with external purchase links.
- `/products/ai-grader` (AI Grader): Multimodal grading form.
- `/services` (Services): Service listings with inquiry form.

6.3.6 Deployment

The backend was deployed to Render.com by configuring `gunicorn` as the WSGI server and `whitenoise` for static file serving. `ALLOWED_HOSTS` and `CORS_ALLOW_ALL_ORIGINS` were configured for the production domain. The frontend was deployed to Vercel by connecting the GitHub repository, with `NEXT_PUBLIC_API_URL` set as an environment variable.

6.4 System Integration and Testing

The system was integration-tested using Django Ninja's auto-generated Swagger UI at `/api/docs`. Each endpoint was tested with representative payloads. The AI Grader was tested with a sample rubric and student answer, validating the structured output format.

6.5 System Output

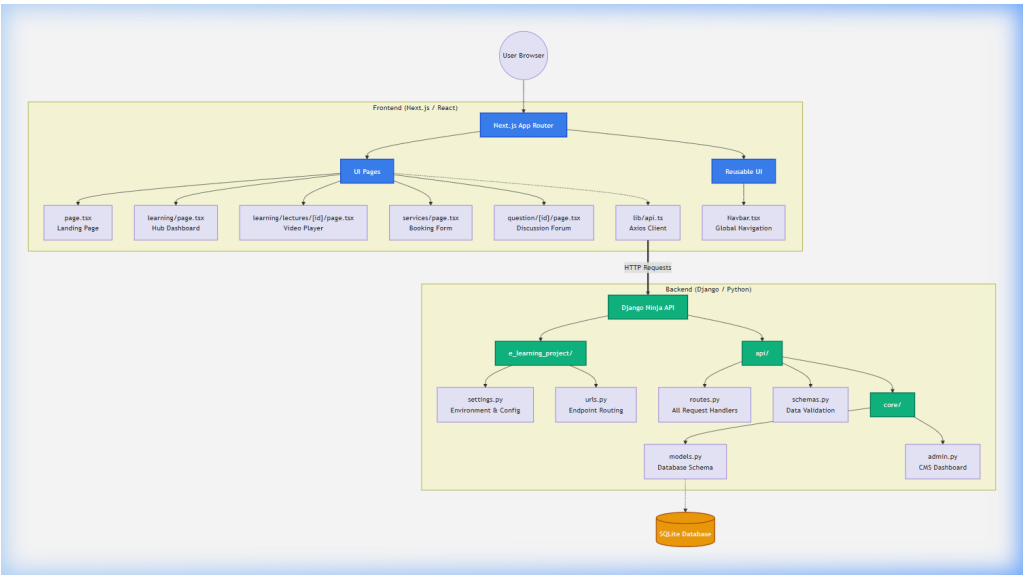


Figure 6.1: NeuralSpark Component Architecture Diagram

6.6 Summary

The implementation of NeuralSpark successfully delivered all seven core modules. The AI Grader endpoint correctly processes text, PDF, and image inputs and returns structured

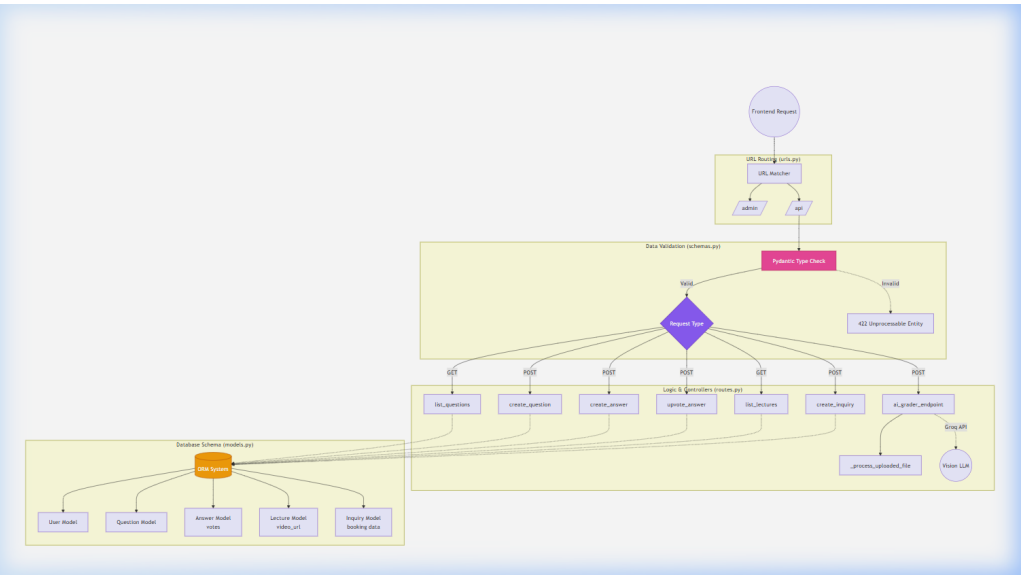


Figure 6.2: NeuralSpark Backend Data Flow Diagram

scores and feedback. The frontend achieves a premium aesthetic through Tailwind CSS and Framer Motion animations.

Chapter 7

Conclusion

NeuralSpark demonstrates that a production-quality, AI-integrated e-learning platform can be built by a single developer using modern, open-source technologies. The project successfully addresses the core problem of automated, fair, and multimodal assignment grading through two complementary pipelines: the Groq Cloud Vision LLM for high-accuracy grading and a locally hosted Transformer model for privacy-first and offline environments.

The Community Q&A Forum, Learning Hub, and PYQ Repository provide a holistic educational ecosystem beyond just grading. The platform's clean decoupled architecture — with a Django Ninja REST API and a standalone Next.js frontend — ensures that each component is independently maintainable and scalable.

All stated objectives of the project have been met:

- A functional full-stack web application built with Django 6 + Next.js 16.
- A multimodal AI grader supporting text, PDF, and image inputs.
- Two independent grading pipelines (cloud and local).
- A complete community forum with voting and moderation.
- A learning module with video lectures and PYQ banks.
- Successful deployment on Vercel and Render.com.

Chapter 8

Future Work

8.1 Real-Time Collaboration

Implementing Django Channels (WebSockets) would enable real-time collaborative learning sessions, live doubt-solving, and real-time peer code review within the forum.

8.2 Advanced Authentication

Replacing the current test-user placeholder with a full JWT-based authentication system (using `djangorestframework-simplejwt`) would enable personalized dashboards, submission history, and reputation-based access controls.

8.3 Mobile Application

A React Native mobile application sharing the same Django Ninja backend would extend NeuralSpark's reach to mobile learners, enabling offline access to PYQs and lecture notes.

8.4 Fine-Tuned Rubric-Aware Model

The local Transformer model can be further fine-tuned on a domain-specific dataset of Computer Science assignment submissions and expert grades to improve its accuracy beyond general-purpose text classification.

8.5 Multi-Language Support

Adding i18n support to the frontend and extending the AI grader to support non-English rubrics (e.g., Hindi, Tamil) would make the platform accessible to a broader student population.

8.6 Analytics Dashboard

An instructor-facing analytics dashboard showing submission trends, average scores, common mistakes, and student performance over time would add significant educational value.

Bibliography

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint arXiv:1810.04805*, 2019.

[2] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] X. Wang et al., "Multimodal Assessment Frameworks for E-Learning," *IEEE Transactions on Learning Technologies*, vol. 15, no. 3, 2022.

[4] M. Porchet, "Comparative Analysis of Python REST API Frameworks," *Software Engineering Journal*, 2023.

[5] Vercel Engineering, "Understanding React Server Components," *Vercel Blog*, 2023. [Online]. Available: https://vercel.com/blog/understanding-react-server-components

[6] Groq Inc., "Groq API Documentation," 2024. [Online]. Available: https://console.groq.com/docs

[7] HuggingFace, "Transformers Library Documentation," 2023. [Online]. Available: https://huggingface.co/docs/transformers

[8] Django Software Foundation, "Django 6.0 Documentation," 2024. [Online]. Available: https://docs.djangoproject.com

[9] O. Vienot, "fpdf2: FPDF for Python," 2023. [Online]. Available: https://py-pdf.github.io/fpdf2/

[10] PyPDF2 Contributors, "PyPDF2 Documentation," 2023. [Online]. Available: https://pypdf2.readthedocs.io